

## 6.2 Character Input and Output

[This section corresponds to K&R Sec. 1.5]

Unless a program can read some input, it's hard to keep it from doing exactly the same thing every time it's run, and thus being rather boring after a while.

The most basic way of reading input is by calling the function `getchar`. `getchar` reads one character from the "standard input," which is usually the user's keyboard, but which can sometimes be redirected by the operating system. `getchar` returns (rather obviously) the character it reads, or, if there are no more characters available, the special value `EOF` ("end of file").

A companion function is `putchar`, which writes one character to the "standard output." (The standard output is, again not surprisingly, usually the user's screen, although it, too, can be redirected. `printf`, like `putchar`, prints to the standard output; in fact, you can imagine that `printf` calls `putchar` to actually print each of the characters it formats.)

Using these two functions, we can write a very basic program to copy the input, a character at a time, to the output:

```
#include <stdio.h>

/* copy input to output */

main()
{
    int c;

    c = getchar();

    while(c != EOF)
    {
        putchar(c);
        c = getchar();
    }

    return 0;
}
```

This code is straightforward, and I encourage you to type it in and try it out. It reads one character, and if it is not the `EOF` code, enters a `while` loop, printing one character and reading another, as long as the character read is not `EOF`. This is a straightforward loop, although there's one mystery surrounding the declaration of the variable `c`: if it holds characters, why is it an `int`?

We said that a `char` variable could hold integers corresponding to character set values, and that an `int` could hold integers of more arbitrary values (up to  $\pm 32767$ ). Since most character sets contain a few hundred characters (nowhere near 32767), an `int` variable can in general comfortably hold all `char` values, and then some. Therefore, there's nothing wrong with declaring `c` as an `int`. But in fact, it's important to do so, because `getchar` can return every character value, *plus* that special, non-character value `EOF`, indicating that there are no more characters. Type `char` is only guaranteed to be able to hold all the character values; it is *not* guaranteed to be able to hold this "no more characters" value without possibly mixing it up with some actual character value. (It's like trying to cram five pounds of books into a four-pound box, or 13 eggs into a carton that holds a dozen.) Therefore, you should always remember to use an `int` for anything you assign `getchar`'s return value to.

When you run the character copying program, and it begins copying its input (your typing) to its

output (your screen), you may find yourself wondering how to stop it. It stops when it receives end-of-file (EOF), but how do you send EOF? The answer depends on what kind of computer you're using. On Unix and Unix-related systems, it's almost always control-D. On MS-DOS machines, it's control-Z followed by the RETURN key. Under Think C on the Macintosh, it's control-D, just like Unix. On other systems, you may have to do some research to learn how to send EOF.

(Note, too, that the character you type to generate an end-of-file condition from the keyboard is *not* the same as the special EOF value returned by `getchar`. The EOF value returned by `getchar` is a code indicating that the input system has detected an end-of-file condition, whether it's reading the keyboard or a file or a magnetic tape or a network connection or anything else. In a disk file, at least, there is not likely to be any character *in* the file corresponding to EOF; as far as your program is concerned, EOF indicates the absence of any more characters to read.)

Another excellent thing to know when doing any kind of programming is how to terminate a runaway program. If a program is running forever waiting for input, you can usually stop it by sending it an end-of-file, as above, but if it's running forever *not* waiting for something, you'll have to take more drastic measures. Under Unix, control-C (or, occasionally, the DELETE key) will terminate the current program, almost no matter what. Under MS-DOS, control-C or control-BREAK will sometimes terminate the current program, but by default MS-DOS only checks for control-C when it's looking for input, so an infinite loop can be unkillable. There's a DOS command,

```
break on
```

which tells DOS to look for control-C more often, and I recommend using this command if you're doing any programming. (If a program is in a really tight infinite loop under MS-DOS, there can be no way of killing it short of rebooting.) On the Mac, try command-period or command-option-ESCAPE.

Finally, don't be disappointed (as I was) the first time you run the character copying program. You'll type a character, and see it on the screen right away, and assume it's your program working, but it's only your computer echoing every key you type, as it always does. When you hit RETURN, a full line of characters is made available to your program. It then zips several times through its loop, reading and printing all the characters in the line in quick succession. In other words, when you run this program, it will probably seem to copy the input a line at a time, rather than a character at a time. You may wonder how a program could instead read a character right away, without waiting for the user to hit RETURN. That's an excellent question, but unfortunately the answer is rather complicated, and beyond the scope of our discussion here. (Among other things, how to read a character right away is one of the things that's not defined by the C language, and it's not defined by any of the standard library functions, either. How to do it depends on which operating system you're using.)

Stylistically, the character-copying program above can be said to have one minor flaw: it contains two calls to `getchar`, one which reads the first character and one which reads (by virtue of the fact that it's in the body of the loop) all the other characters. This seems inelegant and perhaps unnecessary, and it can also be risky: if there were more things going on within the loop, and if we ever changed the way we read characters, it would be easy to change one of the `getchar` calls but forget to change the other one. Is there a way to rewrite the loop so that there is only one call to `getchar`, responsible for reading all the characters? Is there a way to read a character, test it for EOF, and assign it to the variable `c`, all at the same time?

There is. It relies on the fact that the assignment operator, `=`, is just another operator in C. An assignment is not (necessarily) a standalone statement; it is an expression, and it has a value (the value that's assigned to the variable on the left-hand side), and it can therefore participate in a larger, surrounding expression. Therefore, most C programmers would write the character-copying loop like

this:

```
while((c = getchar()) != EOF)
    putchar(c);
```

What does this mean? The function `getchar` is called, as before, and its return value is assigned to the variable `c`. Then the value is immediately compared against the value `EOF`. Finally, the true/false value of the comparison controls the `while` loop: as long as the value is not `EOF`, the loop continues executing, but as soon as an `EOF` is received, no more trips through the loop are taken, and it exits. The net result is that the call to `getchar` happens inside the test at the top of the `while` loop, and doesn't have to be repeated before the loop and within the loop (more on this in a bit).

Stated another way, the syntax of a `while` loop is always

```
while( expression ) ...
```

A comparison (using the `!=` operator) is of course an expression; the syntax is

```
expression != expression
```

And an assignment is an expression; the syntax is

```
expression = expression
```

What we're seeing is just another example of the fact that expressions can be combined with essentially limitless generality and therefore infinite variety. The left-hand side of the `!=` operator (its first *expression*) is the (sub)expression `c = getchar()`, and the combined expression is the *expression* needed by the `while` loop.

The extra parentheses around

```
(c = getchar())
```

are important, and are there because because the *precedence* of the `!=` operator is higher than that of the `=` operator. If we (incorrectly) wrote

```
while(c = getchar() != EOF)                /* WRONG */
```

the compiler would interpret it as

```
while(c = (getchar() != EOF))
```

That is, it would assign the result of the `!=` operator to the variable `c`, which is *not* what we want.

("Precedence" refers to the rules for which operators are applied to their operands in which order, that is, to the rules controlling the default grouping of expressions and subexpressions. For example, the multiplication operator `*` has higher precedence than the addition operator `+`, which means that the expression `a + b * c` is parsed as `a + (b * c)`. We'll have more to say about precedence later.)

The line

```
while((c = getchar()) != EOF)
```

epitomizes the cryptic brevity which C is notorious for. You may find this terseness infuriating (and you're not alone!), and it can certainly be carried too far, but bear with me for a moment while I defend it.

The simple example we've been discussing illustrates the tradeoffs well. We have four things to do:

1. call `getchar`,
2. assign its return value to a variable,
3. test the return value against `EOF`, and
4. process the character (in this case, print it out again).

We can't eliminate any of these steps. We have to assign `getchar`'s value to a variable (we can't just use it directly) because we have to do two different things with it (test, and print). Therefore, compressing the assignment and test into the same line is the only good way of avoiding two distinct calls to `getchar`. You may not agree that the compressed idiom is better for being more compact or easier to read, but the fact that there is now only one call to `getchar` *is* a real virtue.

Don't think that you'll have to write compressed lines like

```
while((c = getchar()) != EOF)
```

right away, or in order to be an ``expert C programmer." But, for better or worse, most experienced C programmers do like to use these idioms (whether they're justified or not), so you'll need to be able to at least recognize and understand them when you're reading other peoples' code.

---

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)